| | |
|---|---|
| **Nombre**: Jaime Sebastian Esparza Leyva | **Matrícula**: 2097304 |
| **Nombre del curso:** Componente de lógica de negocios | **Nombre del profesor**: Miguel Ángel Gómez Marroquín |
| **Modulo Netacad**: NA | **Actividad**: Proyecto Integrador de Aprendizaje (PIA) |
| **Fecha**: 06/11/2026 | |
| **GitHub:** https://github.com/Sebastian-Esp1/PIA-CLN-EQUIPO-3.git | |

## Contenido

### Tabla 1. Contenido del reporte


| Sección | Contenido | Página |
|---|---|---|
| **1** | Resumen y cumplimiento del PIA | 3 |
| **2** | BRD: contexto, necesidades y reglas | 6 |
| **3** | Historias de usuario y Gherkin | 10 |
| **4** | Proceso de negocio (BPMN TO-BE) | 13 |
| **5** | Arquitectura y diseño DDD/UML | 14 |
| **6** | Decisiones de arquitectura (ADR) | 23 |
| **7** | API REST y contrato OpenAPI | 24 |
| **8** | Paquete de datos del docente y su adaptación | 29 |
| **9** | Pruebas, cobertura y calidad | 32 |
| **10** | Matriz de trazabilidad | 35 |
| **11** | Docker, CI/CD y despliegue | 37 |
| **12** | Uso responsable de la IA y limitaciones | 40 |
| **13** | Conclusiones | 42 |

## 1. Resumen y cumplimiento del PIA

Sazón Urbano necesitaba sincronizar pedidos, cocina, ventas e inventario. Este proyecto integrador (PIA) entrega una API REST con la lógica de negocio construida con DDD y arquitectura por capas, derivada del BRD, las historias de usuario, el BPMN y el diseño de las Unidades 2 y 3, y validada con el paquete de datos del docente.

**Tabla 2. Resumen del proyecto**

| Campo | Contenido |
|---|---|
| **Equipo 3** | Jaime Sebastian Esparza Leyva (2097304), Alan Joel Sánchez García (2082464), Jorge Alejandro Jiménez González (2097849) |
| **Tecnología** | Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2 (SQLite en pruebas, PostgreSQL en Docker y despliegue), pytest, Ruff, Docker Compose y GitHub Actions. |
| **Resultados medidos** | 269 pruebas en verde (90 unitarias y 179 de integración; 1 más solo corre con PostgreSQL), cobertura de ramas y líneas de 98.5 % (mínimo 80 %), Ruff sin observaciones, 120 de 120 casos y 20 de 20 transiciones del docente, 25 rutas en OpenAPI. |
| **Patrones** | Aggregates, Value Objects, Domain Services, Specification (AND/OR/NOT), máquina de estados, Domain Events, Repository, Unit of Work, CQRS básico, concurrencia optimista, puertos y adaptadores. |
| **Repositorio y despliegue** | *https://github.com/Sebastian-Esp1/PIA-CLN-EQUIPO-3.git*<br>*https://sazon-urbano-api.onrender.com/docs* |

### Tabla 3. Cumplimiento de los entregables y criterios del PIA

| Requisito del PIA | Estado | Evidencia |
|---|---|---|
| **Reporte final en PDF (BRD, arquitectura, evidencias, conclusiones)** | Cumple | Este documento |
| **docs/BRD.md y docs/architecture/ (ADR y diagramas)** | Cumple | docs/BRD.md, adr.md, 8 diagramas PNG y Mermaid, BPMN 2.0 |
| **docs/api/ con OpenAPI** | Cumple | docs/api/openapi.json (25 rutas) generado por scripts/exportar_openapi.py |
| **src/ con domain, application, infrastructure y api** | Cumple | Secciones 5 y 7 |
| **tests/ con unitarias e integración** | Cumple | Sección 9: 90 unitarias y 179 de integración |
| **.github/workflows/** | Cumple (verde en GitHub Actions) | ci.yml con Ruff, pytest ≥ 80 %, PostgreSQL y Docker |
| **docker-compose.yml con DB, App y Tests; .env.example** | Cumple (verificado en el trabajo docker de GitHub Actions) | Sección 11 |
| **README con URL del repositorio y del despliegue** | Cumple | README.md |
| **Historial de commits** | Cumple | Commits por capa y propósito (git log) |
| **URL de despliegue con Swagger UI** | Cumple (en línea en Render) | https://sazon-urbano-api.onrender.com/docs; render.yaml y pasos en la sección 11 |
| **Cobertura mínima de 80 %** | Cumple: 98.5 % | Sección 9 |

| Requisito del PIA | Estado | Evidencia |
|---|---|---|
| **Trazabilidad BRD → US → Gherkin → BPMN → dominio → código → pruebas** | Cumple | Sección 10 y docs/architecture/trazabilidad.md |
| **Variables de entorno (12-Factor)** | Cumple | config.py, .env.example; sin secretos en el repositorio |

## 2. BRD: contexto, necesidades y reglas

Contexto y problema tomados del BRD de la Unidad 2 (original en docs/requirements/originales_unidad2). Los supuestos SP vienen del informe de la Unidad 3.

### Tabla 4. Contexto, problema y alcance

| Elemento | Descripción |
|---|---|
| **Empresa** | Negocio de alimentos con servicio en mesa, para llevar y a domicilio. |
| **Canales** | Mesa, teléfono, WhatsApp y domicilio. Los pedidos incluyen productos, cantidades y modificaciones. |
| **Estaciones de cocina** | Parrilla, preparación fría, bebidas y empaque. |
| **Problema** | Falta de sincronía entre pedidos, cocina, ventas e inventario: productos agotados no actualizados, órdenes incompletas, diferencias ventas contra inventario, cambios y cancelaciones difíciles de rastrear. |
| **Dentro del alcance** | Pedidos, disponibilidad, comunicación con cocina, estaciones, ingredientes y existencias, cancelaciones, promociones, trazabilidad y consultas de ventas e inventario. |
| **Fuera del alcance** | Nómina, contabilidad, fiscal, proveedores, recursos humanos, marketing y programas de socios. |

### Tabla 5. Necesidades del negocio y cómo las resuelve el código

| ID | Necesidad | Componente |
|---|---|---|
| **N-01** | Control de pedidos | Aggregate Pedido y ServicioDePedidos |
| **N-02** | Comunicación con cocina | Pedido.confirmar, ServicioDeCocina y consulta de órdenes por estación |
| **N-03** | Control de disponibilidad | DisponibilidadProductos y Producto.marcar_agotado |
| **N-04** | Control de inventario | Ingrediente, ConsumoDeIngredientes y comparativo ventas contra inventario |

| ID | Necesidad | Componente |
|---|---|---|
| **N-05** | Trazabilidad | RegistroDeCambio, Cancelacion y consulta de historial |
| **N-06** | Gestión de excepciones | Errores de dominio con código, incidencia por rechazo y cancelación con motivo |
| **N-07** | Información para la gerencia | Lado de lectura: ventas por periodo y comparativo |
| **N-08** | Control de promociones | Promocion y EvaluadorPromociones |

### Tabla 6. Reglas de negocio RN-01 a RN-15

| ID | Regla |
|---|---|
| **RN-01** | Todo pedido tiene al menos un producto, cantidad mayor a cero y canal válido. |
| **RN-02** | Un producto no disponible no se registra; se informa y se registra la incidencia. |
| **RN-03** | Toda modificación o extra queda asociado a su línea. |
| **RN-04** | Cocina recibe el pedido confirmado completo. |
| **RN-05** | Cada producto se prepara en su estación y cada estación ve solo lo suyo. |
| **RN-06** | El pedido está completo solo cuando todas sus líneas están listas. |
| **RN-07** | Solo un pedido activo se cancela, por usuario autorizado y con motivo, usuario, fecha y hora. |
| **RN-08** | Modificar y cancelar dependen del estado del pedido. |
| **RN-09** | Todo cambio conserva usuario, fecha y hora y se consulta en el historial. |
| **RN-10** | La existencia cambia por conteos y movimientos de entrada y salida. |

| ID | Regla |
|---|---|
| **RN-11** | Un ingrediente es crítico cuando su existencia llega al umbral; se identifican los productos afectados. |
| **RN-12** | La disponibilidad depende de las existencias de la receta. |
| **RN-13** | Una promoción aplica solo dentro de su horario y condiciones. |
| **RN-14** | Una venta se registra a partir de un pedido cerrado. |
| **RN-15** | El comparativo muestra diferencias entre inventario esperado y registrado. |

### Tabla 7. Supuestos (SP) y su estado en el PIA

| ID | Supuesto | Estado |
|---|---|---|
| **SP-01** | Estados: EnRegistro, Confirmado, EnPreparación, Listo, Cerrado, Cancelado | Implementado |
| **SP-02** | Modificar en EnRegistro y Confirmado; cancelar en EnRegistro, Confirmado y EnPreparación | Implementado y probado |
| **SP-03** | Umbral crítico por ingrediente (existencia ≤ umbral) | Implementado |
| **SP-04** | Consumo teórico al confirmar, en una sola transacción | Implementado (ADR-03) |
| **SP-05** | Cancelan Encargado y Gerente | Implementado; los demás reciben 403 |
| **SP-06** | No se modelan precios | Revisado: el paquete del docente trae precios, se modelan con Decimal (ADR-08) |
| **SP-07** | Seguimiento del repartidor solo hasta Listo y Cerrado | Sin cambio |

| ID | Supuesto | Estado |
|---|---|---|
| **SP-08** | Cancelar en Confirmado reintegra ingredientes; en EnPreparación es merma | Implementado y probado |
| **SP-09** | Autenticación y roles en infraestructura | Implementado con encabezados (ADR-09) |

## 3. Historias de usuario y Gherkin

Las 15 historias son las de la Unidad 2. Cada una tiene un criterio Gherkin original (Given, When, Then) que aquí se amplió con escenarios de error para que sea ejecutable contra la API. En el PDF original el story mapping de US-02 habla de promociones; por su enunciado y criterio se implementó como modificaciones y extras.

**Tabla 8. Historias de usuario, criterio Gherkin esencial y prueba que las verifica**

| ID | Historia | Criterio de aceptación (Gherkin resumido) | Reglas | Prueba |
|---|---|---|---|---|
| **US-01** | Registrar pedido con productos y cantidades | Dado un pedido en registro, cuando selecciona productos y cantidades, entonces se guarda con su información; cantidad 0 responde 422 | RN-01 | test_iniciar_pedido…; test_cantidad_debe_ser_entero_positivo |
| **US-02** | Agregar modificaciones y extras | Cuando agrego QUITAR sin cebolla a la línea 1, entonces la línea la muestra; tipo BORRAR responde 422 | RN-03 | test_modificaciones_se_asocian… |
| **US-03** | Seleccionar el canal de venta | Mesa, teléfono, WhatsApp o domicilio se guardan en el pedido; MOSTRADOR responde 422 | RN-01 | test_canal_invalido_se_rechaza; test_codigos_http_del_contrato |
| **US-04** | Rechazar productos no disponibles | Producto agotado responde 409 PRODUCTO_NO_DISPONIBLE, no se agrega y deja incidencia | RN-02, RN-12 | test_cu02_producto_agotado… |
| **US-05** | Marcar y reactivar agotados | El encargado agota y reactiva; el mesero recibe 403 | RN-02 | test_cu02_agotar_y_reactivar_producto |
| **US-06** | Enviar el pedido a cocina | Pedido autorizado confirma y descuenta ingredientes; sin autorización, vacío o cliente inactivo responde 409 | RN-04, SP-04 | test_cu01_registrar_y_confirmar… |
| **US-07** | Ver solo mi estación | Consulta PARRILLA muestra la hamburguesa; estación PATIO responde 422 | RN-05 | test_cu03_cocina_prepara_por_estacion |
| **US-08** | Marcar productos listos | El pedido pasa a LISTO solo cuando todas las líneas están listas | RN-06 | test_preparacion_por_linea… |

| ID | Historia | Criterio de aceptación (Gherkin resumido) | Reglas | Prueba |
|---|---|---|---|---|
| **US-09** | Registrar existencias | Entrada de 4 y salida de 1 cambian la existencia en 3; salida excesiva responde 409 | RN-10 | test_entrada_y_salida…; test_cu05_* |
| **US-10** | Detectar ingredientes críticos | Al cruzar el umbral se marca crítico, con un solo evento y los productos afectados | RN-11, SP-03 | test_es_critico_es_derivado… |
| **US-11** | Cancelar con motivo | Confirmado se cancela con motivo y reintegra; sin motivo 422; Listo o Cerrado 409; mesero 403 | RN-07, RN-08 | test_cu04_*; test_no_se_cancela… |
| **US-12** | Consultar historial | Acciones en orden con usuario y fecha; no se edita | RN-09 | test_historial_solo_crece… |
| **US-13** | Ventas por periodo | Pedido cerrado genera una sola venta; rango invertido 400; mesero 403 | RN-14 | test_reportes_de_ventas… |
| **US-14** | Comparar ventas con inventario | El comparativo da consumo esperado, registrado y diferencia por ingrediente | RN-15 | test_reportes_de_ventas… |
| **US-15** | Promociones con horario | Gerente configura promoción 00:00 a 23:59 con total mínimo 100 y 10 % aplica descuento 58.00 sobre 580; horario inválido 422 | RN-13 | test_cu01_promocion_vigente… |

**Tabla 9. Ejemplo de criterio Gherkin completo (US-11)**

| Código |
| --- |

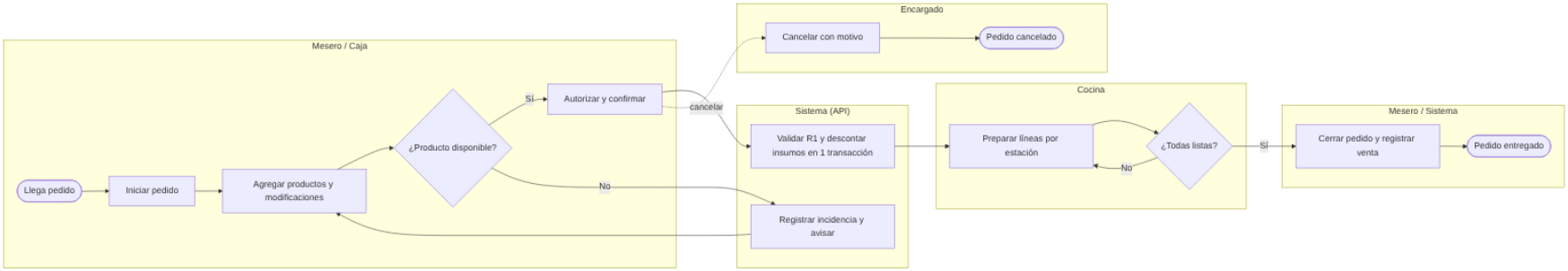
```
Escenario: Cancelación válida
  Dado un pedido confirmado
  Cuando el encargado lo cancela con motivo "cliente se fue"
  Entonces queda CANCELADO con motivo, usuario, fecha y hora
  Y se reintegran los ingredientes consumidos

Escenario: Sin motivo         -> 422 MOTIVO_REQUERIDO
Escenario: Estado Listo o Cerrado -> 409 CANCELACION_NO_PERMITIDA
Escenario: Mesero intenta cancelar -> 403 NO_AUTORIZADO
```

## 4. Proceso de negocio (BPMN TO-BE)

El BPMN AS-IS de la Unidad 2 (carriles Cliente, Mesero/Caja, Cocina, Repartidor, Encargado y Gerente; archivo docs/architecture/bpmn_as_is_unidad2.svg) no incluía cancelaciones, historial ni promociones, y su tramo de entrega con repartidor queda fuera del detalle (SP-07). El proceso TO-BE que implementa la API los integra. El archivo BPMN 2.0 con diagrama está en docs/architecture/bpmn_to_be.bpmn y se puede abrir en bpmn.io o Camunda Modeler.

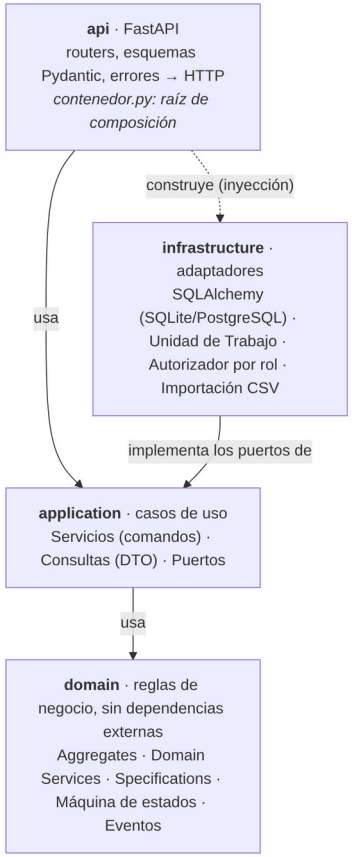
**Tabla 10. Proceso TO-BE del pedido**



## 5. Arquitectura y diseño DDD/UML

Arquitectura en cuatro capas con dependencias hacia el dominio. La capa api solo traduce HTTP; application orquesta casos de uso y define puertos; infrastructure implementa los puertos; domain no depende de nada.

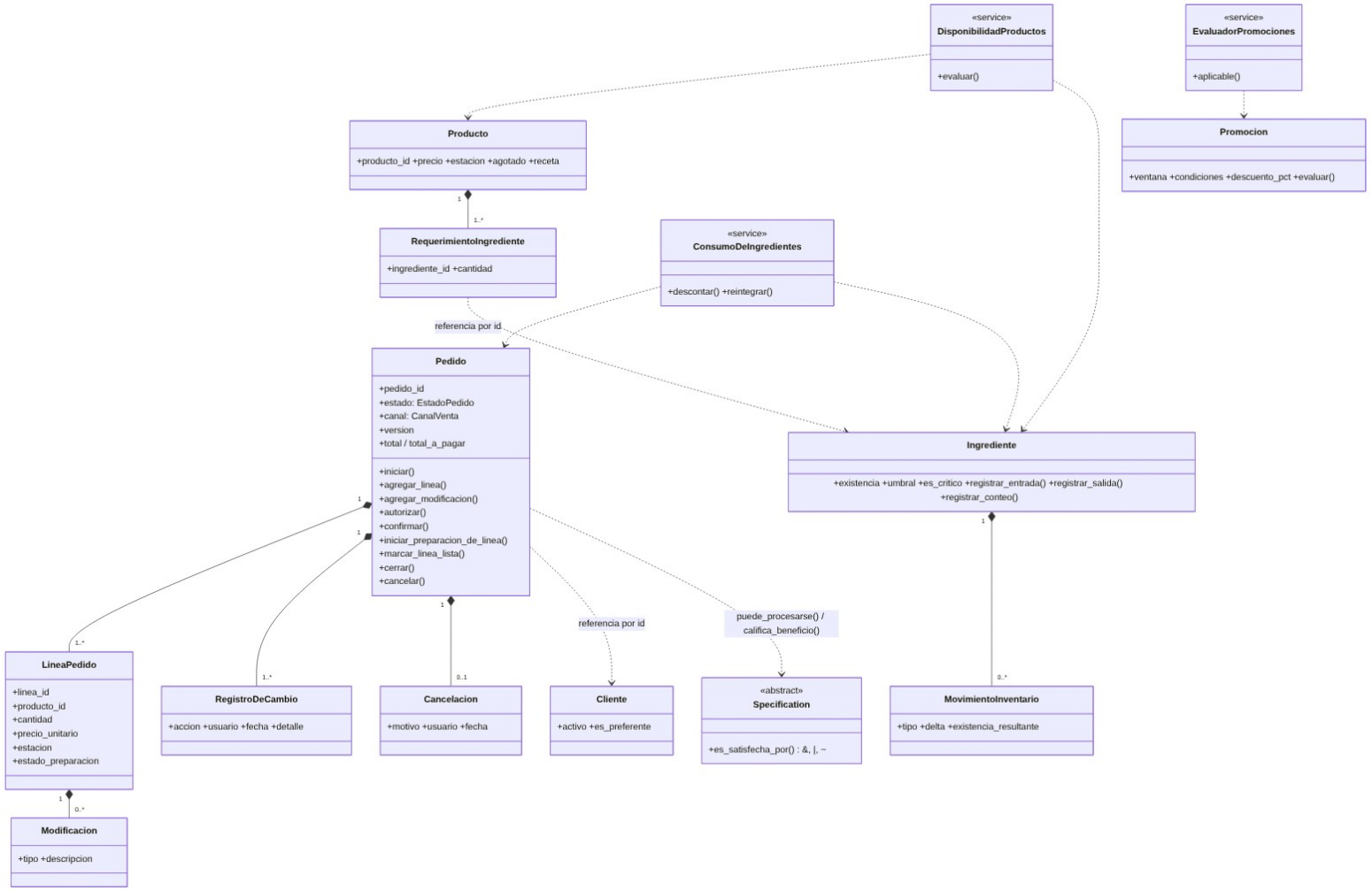
**Tabla 11. Capas y dependencias**



## 5.1 Modelo de dominio (UML)

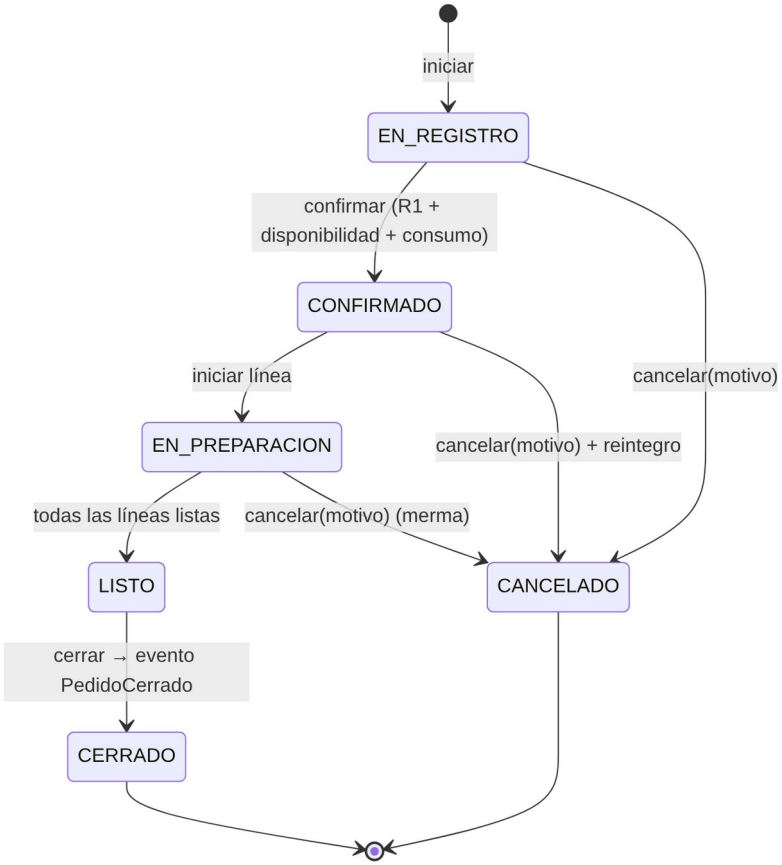
**Tabla 12. Diagrama de clases del dominio**

Reporte

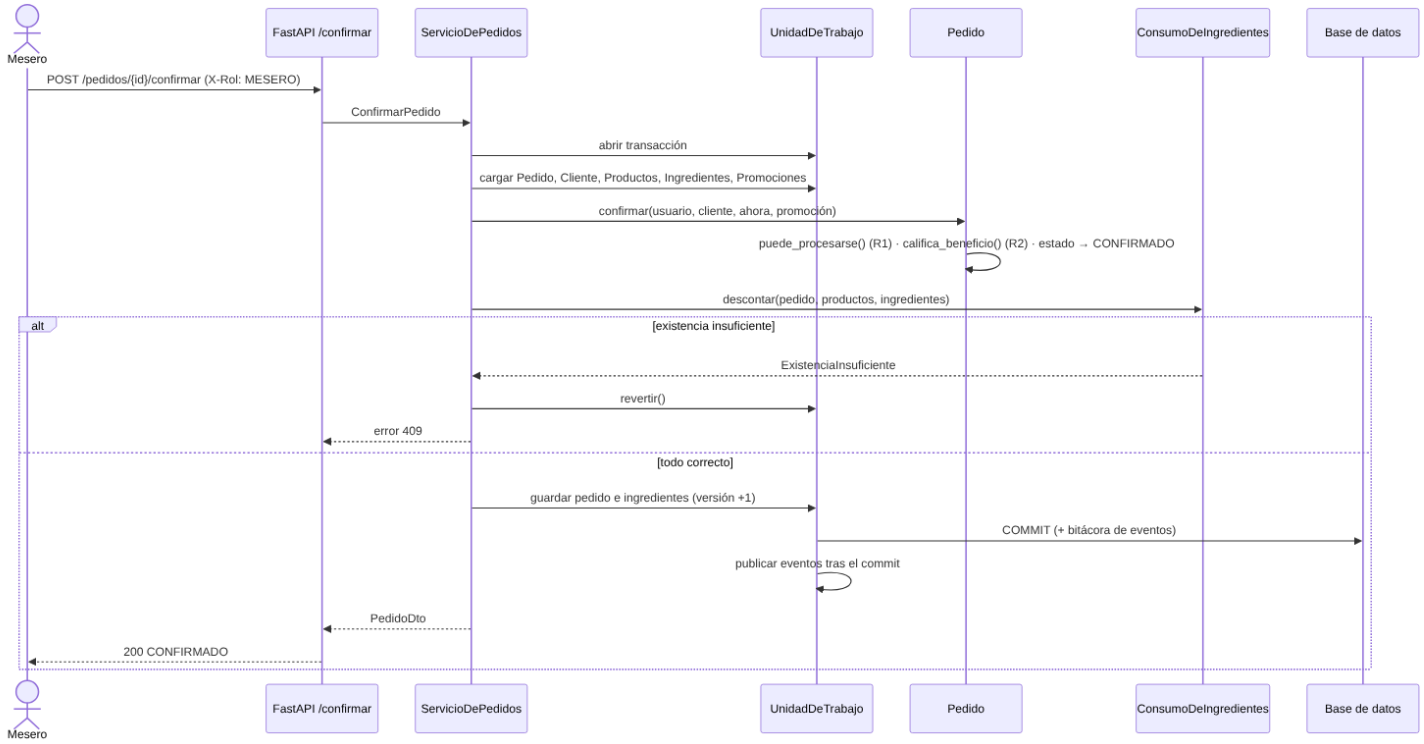

«service»
DisponibilidadProductos
+evaluar()
«service»
EvaluadorPromociones
+aplicable()
Producto
+producto_id +precio +estacion +agotado +receta
Promocion
+ventana +condiciones +descuento_pct +evaluar()
RequerimientoIngrediente
+ingrediente_id +cantidad
«service»
ConsumoDeIngredientes
+descontar() +reintegrar()
referencia por id
Pedido
+pedido_id
+estado: EstadoPedido
+canal: CanalVenta
+version
+total / total_a_pagar
+iniciar()
+agregar_linea()
+agregar_modificacion()
+autorizar()
+confirmar()
+iniciar_preparacion_de_linea()
+marcar_linea_lista()
+cerrar()
+cancelar()
Ingrediente
+existencia +umbral +es_critico +registrar_entrada() +registrar_salida()
+registrar_conteo()
puede_procesarse() /
califica_beneficio()
referencia por id
LineaPedido
+linea_id
+producto_id
+cantidad
+precio_unitario
+estacion
+estado_preparacion
RegistroDeCambio
+accion +usuario +fecha +detalle
Cancelacion
+motivo +usuario +fecha
Cliente
+activo +es_preferente
«abstract»
Specification
+es_satisfecha_por() : &, |, ~
MovimientoInventario
+tipo +delta +existencia_resultante
Modificacion
+tipo +descripcion

## 5.2 Estados del pedido y secuencias

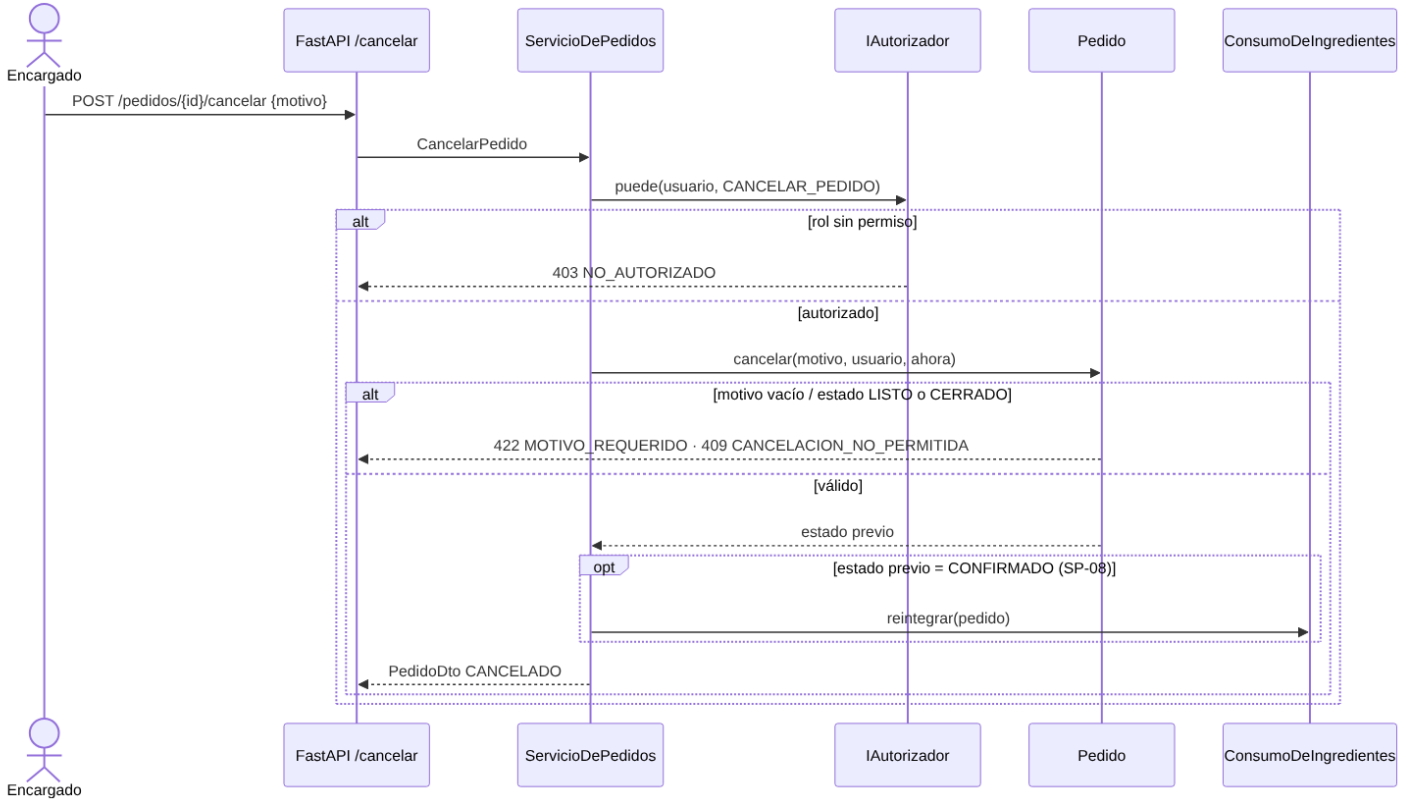
**Tabla 13. Máquina de estados del Pedido**



## Tabla 14. Secuencia de confirmar pedido (CU-01)

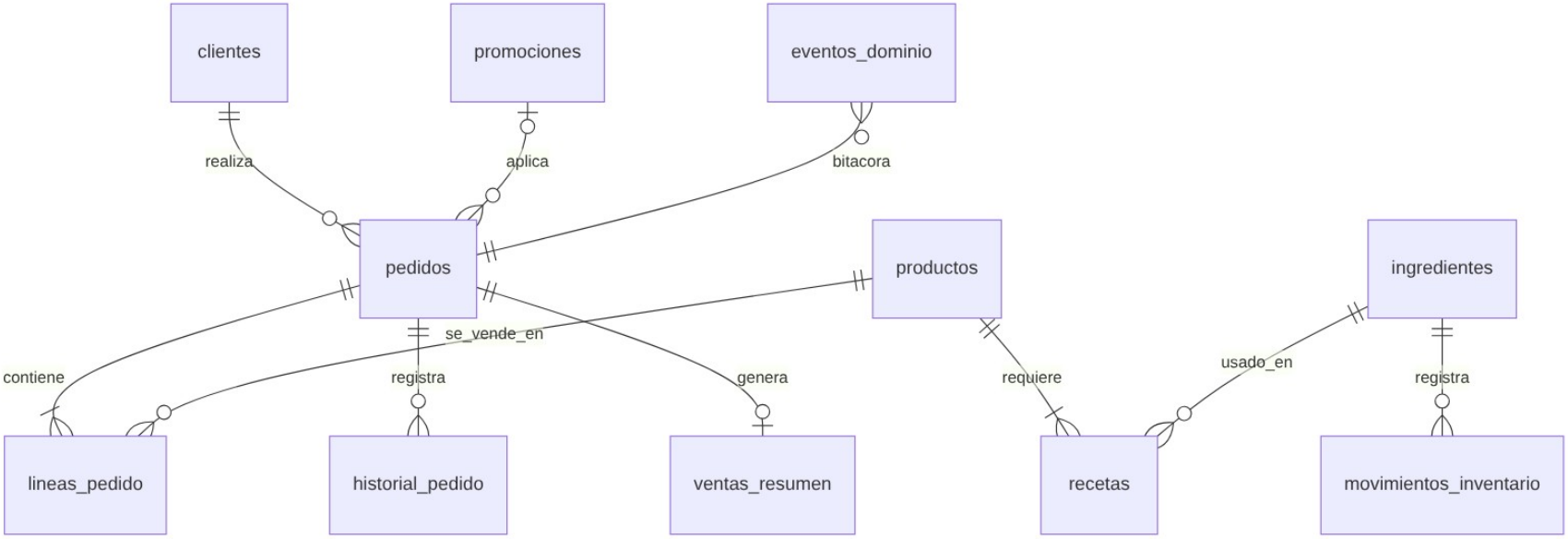


## Tabla 15. Secuencia de cancelar pedido (CU-04)

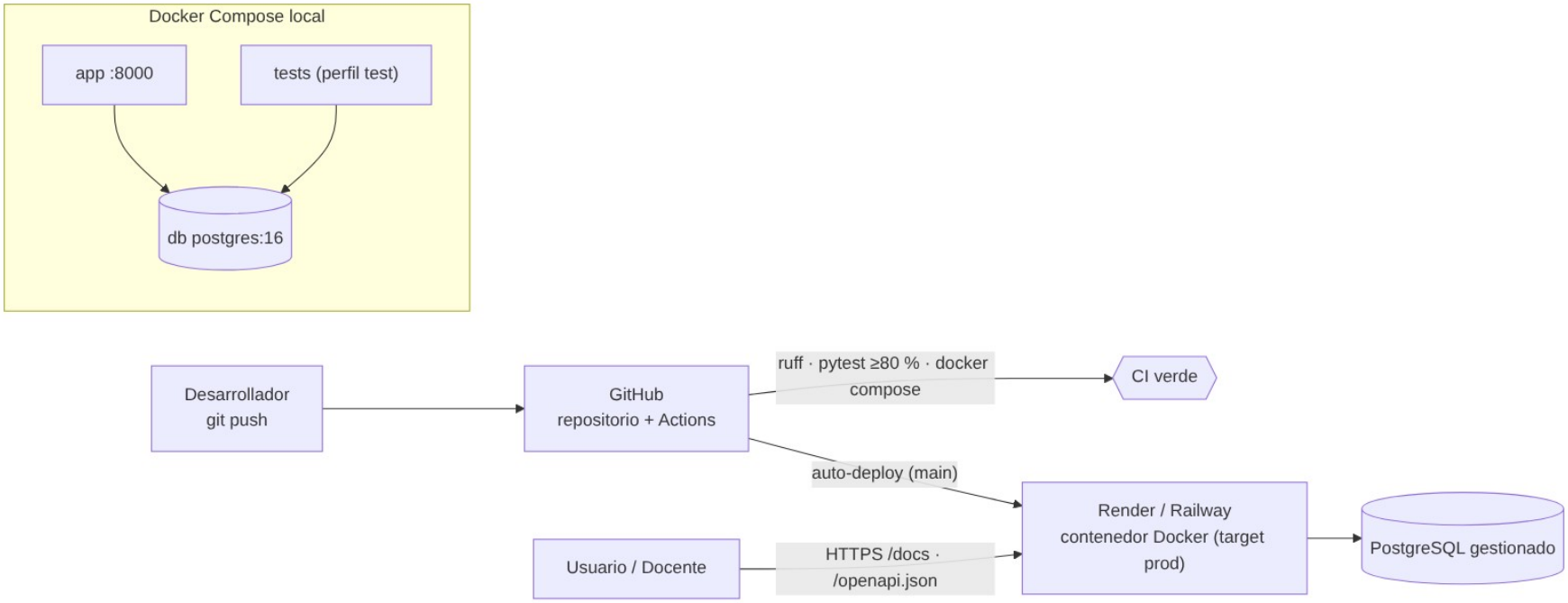


## 5.3 Persistencia y despliegue

**Tabla 16. Modelo entidad-relación**



## Tabla 17. Despliegue




## Tabla 18. Patrones de diseño aplicados y su evidencia

| Patrón o principio | Dónde | Prueba |
|---|---|---|
| **Aggregate Pedido (líneas, historial, cancelación)** | src/domain/entities.py | test_pedido.py (30 pruebas) |

| Patrón o principio | Dónde | Prueba |
|---|---|---|
| **Specification AND/OR/NOT** | src/domain/specifications.py: puede_procesarse y califica_beneficio | test_specifications.py; 120 casos del docente |
| **Máquina de estados** | src/domain/state_machine.py | test_transiciones_permitidas e invalidas; 20 transiciones |
| **Domain Events (15 eventos)** | src/domain/events.py | test_eventos_se_guardan_en_bitacora… |
| **Domain Services** | src/domain/services.py | test_ingrediente_producto_promocion.py |
| **Repository y Unit of Work** | src/infrastructure/repositorios.py y unidad_de_trabajo.py | test_rollback_no_deja_cambios_parciales |
| **Concurrencia optimista** | Columna version en pedidos | test_concurrencia_optimista… |
| **CQRS básico** | commands.py, servicios.py y queries.py | test_api_flujos.py |
| **Inversión de dependencias** | src/application/ports.py y api/contenedor.py | RelojFijo y contenedor en tests/conftest.py |

## 6. Decisiones de arquitectura (ADR)

Las ADR-01 a ADR-04 vienen de la Unidad 3 (estaban en estado Propuesta) y ahora están Aceptadas porque quedaron implementadas y probadas. Las ADR-05 a ADR-09 son nuevas.

**Tabla 19. Registro de decisiones**

| ADR | Decisión | Motivo | Consecuencia |
|---|---|---|---|
| **01** | Domain Model como patrón principal | Reglas que se repiten entre casos de uso (RN-02 a RN-13) | Reglas en un solo lugar; más código inicial |
| **02** | Pedido como un único Aggregate | Pedido completo solo si todas las líneas están listas (RN-06) | Posibles conflictos de versión, resueltos con control optimista |
| **03** | Consumo de ingredientes al confirmar, en una transacción | Evitar vender sin existencia y diferencias ventas contra inventario | Rompe un Aggregate por transacción de forma consciente |
| **04** | CQRS básico sin bases separadas | Lecturas de gerencia y cocina sin cargar Aggregates | Resumen de ventas depende del evento PedidoCerrado; actualizador idempotente |
| **05** | Python, FastAPI y SQLAlchemy en lugar de C# | OpenAPI automático, despliegue gratuito y experiencia del equipo | Se conservan los nombres de la Unidad 3 |
| **06** | Capas con inversión de dependencias | Probar el dominio sin base de datos ni HTTP | Raíz de composición en contenedor.py |
| **07** | Adaptar el paquete del docente al modelo propio | El LEEME exige documentar diferencias y no reemplazar requisitos | Oráculos independientes calculados desde el CSV |
| **08** | Precios con Decimal (revisión de SP-06) | El paquete trae precios en MXN y R2 compara con 1500 | Importes como cadena en JSON |
| **09** | Roles con encabezados y API key opcional | Demostrar autorización sin proveedor de identidad | No es autenticación criptográfica; en producción real iría JWT u OIDC |

## 7. API REST y contrato OpenAPI

El contrato OpenAPI se genera desde el código (docs/api/openapi.json) y se publica en /docs (Swagger UI) y /openapi.json. Los roles viajan en los encabezados X-Usuario y X-Rol.

**Tabla 20. Endpoints de la API (25 rutas)**

| Método | Ruta | Descripción |
|---|---|---|
| GET | /health | Estado del servicio |
| POST | /pedidos | Iniciar pedido (CU-01) |
| GET | /pedidos | Listar pedidos |
| GET | /pedidos/{pedido_id} | Consultar un pedido |
| POST | /pedidos/{pedido_id}/lineas | Agregar producto (CU-01 / CU-02: rechaza si no hay disponibilidad) |
| POST | /pedidos/{pedido_id}/lineas/{linea_id}/modificaciones | Agregar modificación o extra a una línea (RN-03) |
| POST | /pedidos/{pedido_id}/autorizar | Autorizar pedido |
| POST | /pedidos/{pedido_id}/confirmar | Confirmar pedido y enviarlo a cocina (CU-01) |
| POST | /pedidos/{pedido_id}/cancelar | Cancelar pedido con motivo (CU-04) |
| POST | /pedidos/{pedido_id}/cerrar | Cerrar (entregar) pedido |
| GET | /pedidos/{pedido_id}/historial | Historial de cambios del pedido (US-12) |
| GET | /cocina/estaciones/{estacion}/ordenes | Órdenes de una estación (US-07) |
| POST | /cocina/pedidos/{pedido_id}/lineas/{linea_id}/iniciar | Iniciar preparación de una línea |
| POST | /cocina/pedidos/{pedido_id}/lineas/{linea_id}/listo | Marcar línea como lista (US-08) |
| GET | /productos/disponibilidad | Disponibilidad de productos (US-04) |

| Método | Ruta | Descripción |
|---|---|---|
| POST | /productos/{producto_id}/agotar | Marcar agotado (US-05) |
| POST | /productos/{producto_id}/reactivar | Reactivar producto (US-05) |
| GET | /inventario/ingredientes | Existencias |
| GET | /inventario/criticos | Ingredientes críticos (US-10) |
| POST | /inventario/ingredientes/{ingrediente_id}/movimientos | Registrar entrada o salida (US-09) |
| POST | /inventario/ingredientes/{ingrediente_id}/conteo | Registrar conteo físico (US-09) |
| PUT | /inventario/ingredientes/{ingrediente_id}/umbral | Definir umbral crítico (SP-03) |
| POST | /promociones | Configurar promoción |
| GET | /reportes/ventas | Ventas por periodo (US-13) |
| GET | /reportes/comparativo-inventario | Comparativo ventas vs. inventario (US-14) |
| POST | /admin/importar-csv | Cargar el paquete de datos del docente (idempotente) |

## Tabla 21. Roles y permisos (SP-05)

| Rol | Permisos |
|---|---|
| **MESERO** | Registrar pedidos, agregar productos, autorizar, confirmar y cerrar |
| **COCINERO** | Operar cocina: iniciar y marcar líneas listas, consultar estaciones |
| **ENCARGADO** | Lo anterior más cancelar, inventario, catálogo, promociones, reportes y administración de datos |
| **GERENTE** | Todos los permisos |

### Tabla 22. Códigos HTTP del contrato

| Código | Uso |
|---|---|
| **200** | Acción correcta (confirmar, cancelar, consultar) |
| **201** | Creación de pedido, línea, modificación o promoción |
| **400** | Rango de fechas invertido o rol inválido |
| **401** | API key ausente o incorrecta (solo si el servidor define API_KEY) |
| **403** | El rol no tiene permiso (NO_AUTORIZADO) |
| **404** | Pedido, producto, cliente o ingrediente inexistente (NO_ENCONTRADO) |
| **409** | Regla de negocio, estado incompatible o concurrencia: PRODUCTO_NO_DISPONIBLE, PEDIDO_NO_PROCESABLE, PEDIDO_NO_MODIFICABLE, TRANSICION_INVALIDA, CANCELACION_NO_PERMITIDA, EXISTENCIA_INSUFICIENTE, CONFLICTO_DE_CONCURRENCIA |
| **422** | Datos inválidos: DATO_INVALIDO, MOTIVO_REQUERIDO o cuerpo que no cumple el esquema |

## Tabla 23. Demostración real de la API en ejecución (uvicorn con datos del docente)

```
$ curl -s localhost:8765/health
{"estado":"ok"}

$ POST /pedidos  {"canal":"MESA","cliente_id":"PED-C001"}   -> 201
pedido_id: PED-0F2307F9

$ POST /pedidos/PED-0F2307F9/lineas  {"producto_id":"PED-P011","cantidad":1}
{"codigo":"PRODUCTO_NO_DISPONIBLE","mensaje":"Pasta 11 está agotado"}
HTTP 409

$ POST .../lineas  {"producto_id":"PED-P001","cantidad":2}
HTTP 201
POST .../autorizar -> HTTP 200

$ POST .../confirmar
{
 "pedido_id": "PED-0F2307F9",
 "estado": "CONFIRMADO",
 "total": "580.00",
 "descuento": "0.00",
 "total_a_pagar": "580.00"
}
HTTP200

$ POST .../confirmar (segunda vez)
{"codigo":"TRANSICION_INVALIDA","mensaje":"El pedido no puede pasar de CONFIRMADO a CONFIRMADO."}
HTTP 409

$ POST .../cancelar con rol MESERO
{"codigo":"NO_AUTORIZADO","mensaje":"El rol MESERO no puede realizar CANCELAR_PEDIDO."}
HTTP 403

$ POST .../cancelar con rol ENCARGADO
CANCELADO {'motivo': 'cliente se fue', 'usuario': 'enc1', 'fecha': '2026-11-05T23:38:47'}

$ GET .../historial
2026-11-05T23:38:47 demo     Pedido iniciado | canal=MESA
2026-11-05T23:38:47 demo     Producto rechazado por disponibilidad | PED-P011: Pasta 11 está agotado
2026-11-05T23:38:47 demo     Línea agregada | PED-P001 x2
2026-11-05T23:38:47 demo     Pedido autorizado |
2026-11-05T23:38:47 demo     Pedido confirmado | enviado a cocina e inventario
2026-11-05T23:38:47 enc1     Pedido cancelado | motivo: cliente se fue

$ GET /inventario/ingredientes/ING-PED-P001 (tras confirmar y cancelar: reintegrado)
{'ingrediente_id': 'ING-PED-P001', 'nombre': 'Insumo base de Hamburguesa 01', 'existencia': '11.000', 'umbral': '3.000', 'es_critico': False}
```

*Nota: la captura de Swagger UI (/docs) debe tomarse desde el despliegue público o con la API corriendo en tu equipo; aquí se muestran las respuestas reales de la misma API.*

| Evidencia | Cómo obtenerla | Captura |
|---|---|---|
| **Swagger UI del despliegue** | Abrir https://<tu-servicio>/docs y expandir POST /pedidos/{pedido_id}/confirmar |  |

## 8. Paquete de datos del docente y su adaptación

El LEEME del paquete aclara que su flujo y sus reglas son un modelo de pruebas propuesto y que el PIA exige derivar la implementación del BRD propio. Por eso se documentaron las diferencias y se adaptaron fixtures y oráculos.

**Tabla 24. Diferencias entre el paquete del docente y el modelo del equipo**

| Tema | Paquete del docente | Modelo del equipo | Adaptación |
|---|---|---|---|
| **Actor** | Comensal activo o preferente | Cliente y usuarios con rol | Entidad Cliente; los roles son usuarios del sistema |
| **Recurso** | Platillo con precio, stock y habilitado | Producto con receta y estación | Receta de un insumo base por producto; stock es la existencia inicial; habilitado = 0 es agotado |
| **Estados** | CREADO, ACEPTADO, EN_PREPARACION, ENTREGADO, CANCELADO | EN_REGISTRO, CONFIRMADO, EN_PREPARACION, LISTO, CERRADO, CANCELADO | Se mantiene la máquina propia; equivalencias CREADO ≈ EN_REGISTRO, ACEPTADO ≈ CONFIRMADO, ENTREGADO ≈ CERRADO |
| **R1 puede_procesarse** | Actor activo y línea y autorización | RN-01 y RN-04 | Specification AND y precondición de confirmar |
| **R2 beneficio_referencia** | Preferente, total ≥ 1500 y sin promoción | RN-13 (promoción) | Specification AND/NOT; marca beneficio_aplicado; no sustituye la promoción propia |
| **Precios** | MXN | SP-06: sin precios | Revisión de SP-06 con Decimal |
| **Acciones de transición** | aprobar, completar, cancelar, repetir_aprobacion, saltar_estado | confirmar, preparar, listo, cerrar, cancelar | completar recorre confirmar, preparar y listo de todas las líneas y cerrar; saltar_estado es cerrar desde EN_REGISTRO |

**Tabla 25. Resultados contra el oráculo independiente**

| Conjunto | Resultado | Cómo se verificó |
|---|---|---|
| **120 casos de prueba (60 validar_operacion, 60 evaluar_beneficio)** | 120 de 120 | El oráculo recalcula desde los CSV, sin usar el dominio: total = suma precio × cantidad con Decimal, R1 y R2. Hay casos positivos y negativos de cada regla. |
| **20 transiciones (4 aprobar, 4 completar, 4 cancelar, 4 repetir_aprobacion, 4 saltar_estado)** | 20 de 20 | Se ejecutan por la API. Aprobar y completar fallan con 409 si el pedido no es procesable o el producto está deshabilitado o sin existencia; repetir_aprobacion nunca aprueba dos veces; saltar_estado da 409 TRANSICION_INVALIDA. |
| **Importación de datos** | 40 clientes, 20 productos, 60 pedidos, 117 líneas | Idempotente: una segunda importación no duplica nada. |

**Tabla 26. Ejecución real de las pruebas del paquete del docente**

```
$ pytest tests/integration/test_paquete_docente.py -v
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-T001] PASSED
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-B001] PASSED
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-T002] PASSED
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-B002] PASSED
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-T003] PASSED
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-B003] PASSED
...
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-B059] PASSED
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-T060] PASSED
tests/integration/test_paquete_docente.py::test_casos_de_prueba_del_docente[PED-B060] PASSED

tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S001] PASSED
tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S002] PASSED
tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S003] PASSED
tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S004] PASSED
tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S005] PASSED
...
tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S018] PASSED
tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S019] PASSED
tests/integration/test_paquete_docente.py::test_transiciones_del_docente[PED-S020] PASSED

120 casos de prueba y 20 transiciones del docente: PASSED
```

## 9. Pruebas, cobertura y calidad

**Tabla 27. Resumen de pruebas y calidad**

| Indicador | Resultado | Detalle |
|---|---|---|
| **Pruebas unitarias** | 90 | Value Objects, máquina de estados, Specifications, Pedido, Ingrediente, Producto, Promoción y Domain Services, sin base de datos |
| **Pruebas de integración** | 179 + 1 | API completa, repositorios, concurrencia, rollback, eventos, configuración y paquete del docente; la prueba sobre PostgreSQL corre solo con TEST_DATABASE_URL (CI y Docker) |
| **Cobertura (líneas y ramas)** | 98.51 % | Mínimo exigido 80 %; el pipeline falla por debajo de ese valor |
| **Análisis estático** | Ruff sin observaciones | Reglas E, F, I, UP y B |
| **Defecto encontrado al probar** | 1 | Choque de nombres entre un parámetro y un dato del evento en el método interno que emite eventos (TypeError al agregar modificaciones); se corrigió y quedó cubierto por test_modificaciones_se_asocian_a_la_linea |

## Tabla 28. Cobertura por módulo (salida real de pytest-cov)

```
$ pytest --cov=src --cov-branch --cov-fail-under=80
Name                                      Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------------
src/__init__.py                               0      0      0      0   100%
src/api/__init__.py                           0      0      0      0   100%
src/api/aplicacion.py                       137      1     12      1    99%   364
src/api/contenedor.py                        36      0      0      0   100%
src/api/esquemas.py                         123      0      0      0   100%
src/api/principal.py                          2      0      0      0   100%
src/application/__init__.py                   0      0      0      0   100%
src/application/commands.py                  78      0      0      0   100%
src/application/dtos.py                      85      0      0      0   100%
src/application/ports.py                     41      0      0      0   100%
src/application/queries.py                   39      0      4      0   100%
src/application/servicios.py                227      3     36      3    98%   169, 312, 345
src/domain/__init__.py                        0      0      0      0   100%
src/domain/entities.py                      272      1     62      1    99%   73
src/domain/errors.py                         39      0      0      0   100%
src/domain/events.py                         26      0     30      0   100%
src/domain/services.py                       36      0     18      0   100%
src/domain/specifications.py                 83      0      0      0   100%
src/domain/state_machine.py                  32      0      2      0   100%
src/domain/value_objects.py                  92      0     10      0   100%
src/infrastructure/__init__.py                0      0      0      0   100%
src/infrastructure/base_de_datos.py          15      1      4      1    89%   19
src/infrastructure/config.py                 22      0      6      0   100%
src/infrastructure/consultas.py              85      0     10      0   100%
src/infrastructure/importacion_csv.py        58      0     18      0   100%
src/infrastructure/modelos.py               102      0      0      0   100%
src/infrastructure/repositorios.py          140      8     36      4    91%   258, 290, 310-312, 318-320
src/infrastructure/seguridad.py               5      0      0      0   100%
src/infrastructure/unidad_de_trabajo.py      64      3     10      0    96%   68-70
-------------------------------------------------------------------------------------
TOTAL                                      1839     17    258     10    99%
Required test coverage of 80% reached. Total coverage: 98.52%
================== 267 passed, 1 skipped, 1 warning in 47.81s ==================
```

### Tabla 29. Análisis estático

```
$ ruff check .
All checks passed!
```

### Tabla 30. Pruebas de integración más relevantes

| Prueba | Qué demuestra |
|---|---|
| **test_cu01_confirmacion_con_existencia_insuficiente_hace_rollback** | Si falta existencia, el pedido sigue EN_REGISTRO y el inventario no cambia (ADR-03) |
| **test_concurrencia_optimista_rechaza_la_segunda_escritura_vieja** | Dos usuarios que modifican la misma versión: el segundo recibe conflicto (ADR-02) |
| **test_cu04_cancelar_confirmado_reintegra_existencias_y_exige_rol** | Cancelar reintegra inventario (SP-08) y el mesero recibe 403 |
| **test_la_venta_se_registra_una_sola_vez** | El suscriptor de PedidoCerrado es idempotente (ADR-04) |
| **test_matriz_de_roles** | Cada rol solo accede a lo permitido (SP-05) |
| **test_codigos_http_del_contrato** | 201, 200, 404, 422, 400 y 403 según el contrato |
| **test_api_key_opcional_protege_la_api** | 401 sin clave y 201 con ella |

## 10. Matriz de trazabilidad

Cada necesidad del BRD llega a una regla, una historia, un caso de uso, un componente y al menos una prueba automática. La versión completa está en docs/architecture/trazabilidad.md.

**Tabla 31. Matriz de trazabilidad**

| Necesidad | Regla | Historia | Caso de uso | Componente | Prueba |
|---|---|---|---|---|---|
| **N-01** | RN-01 | US-01, US-03 | CU-01 | Pedido.iniciar, agregar_linea | test_cantidad_debe_ser_entero_positivo; test_codigos_http_del_contrato |
| **N-01** | RN-03 | US-02 | CU-01 | Pedido.agregar_modificacion | test_modificaciones_se_asocian_a_la_linea |
| **N-02** | RN-04, RN-08 | US-06 | CU-01 | Pedido.confirmar, puede_procesarse() | test_confirmar_exige_regla_r1… |
| **N-02** | RN-05 | US-07 | CU-03 | ServicioDeCocina, consulta por estación | test_cu03_cocina_prepara_por_estacion… |
| **N-02** | RN-06 | US-08 | CU-03 | Pedido.marcar_linea_lista | test_preparacion_por_linea_y_pedido_listo… |
| **N-03** | RN-02, RN-12 | US-04, US-05 | CU-02 | DisponibilidadProductos, Producto | test_disponibilidad_considera…; test_cu02_* |
| **N-04** | RN-10 | US-09 | CU-05 | Ingrediente, MovimientoInventario | test_entrada_y_salida…; test_cu05_* |
| **N-04** | RN-11 | US-10 | CU-05 | UmbralCritico, IngredienteMarcadoCritico | test_es_critico_es_derivado… |
| **N-04** | RN-12, SP-04 | US-06 | CU-01 | ConsumoDeIngredientes, UnidadDeTrabajo | test_consumo_descuenta_y_reintegra…; test_rollback_no_deja_cambios_parciales |
| **N-05** | RN-07 | US-11 | CU-04 | Pedido.cancelar, AutorizadorPorRol | test_cancelar_exige_motivo…; test_cu04_* |
| **N-05** | RN-09 | US-12 | CU-04 | RegistroDeCambio, historial | test_historial_solo_crece… |
| **N-06** | RN-08, SP-02 | US-04, US-11 | CU-02, CU-04 | EstadoPedido, MaquinaDeEstados | test_transiciones_permitidas; test_transiciones_invalidas |

| Necesidad | Regla | Historia | Caso de uso | Componente | Prueba |
|---|---|---|---|---|---|
| **N-07** | RN-14 | US-13 | — | VentaRow, ActualizadorDeVentas | test_reportes_de_ventas…; test_la_venta_se_registra_una_sola_vez |
| **N-07** | RN-15 | US-14 | — | comparativo de inventario | test_reportes_de_ventas_y_comparativo |
| **N-08** | RN-13 | US-15 | CU-01 | Promocion, EvaluadorPromociones | test_promocion_aplica_solo_vigente…; test_cu01_promocion_vigente… |
| **Docente** | R1 y R2 | — | — | Specifications | test_casos_de_prueba_del_docente (120) |
| **Docente** | Transiciones | — | — | MAQUINA_PEDIDO y API | test_transiciones_del_docente (20) |

## 11. Docker, CI/CD y despliegue

### Tabla 32. Archivos de despliegue y automatización

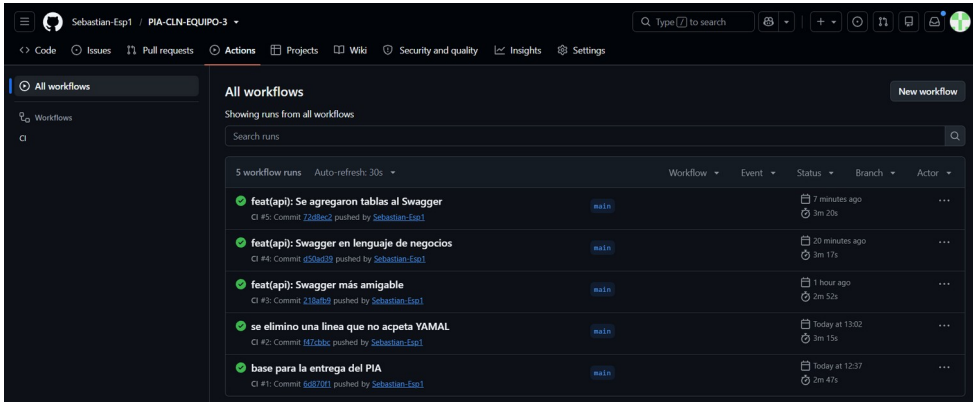
| Archivo | Función |
|---|---|
| **Dockerfile** | Tres etapas: base, test (con Ruff y pytest) y prod (usuario sin privilegios y healthcheck) |
| **docker-compose.yml** | Servicios db (PostgreSQL 16 con healthcheck), app (API en el puerto 8000) y tests (perfil test) |
| **.env.example** | Variables de entorno sin secretos (12-Factor) |
| **.github/workflows/ci.yml** | Ruff, pytest con cobertura ≥ 80 % en Python 3.11, 3.12 y 3.13 con servicio PostgreSQL; exporta OpenAPI; trabajo Docker con compose config, build, pruebas en contenedores y prueba de humo |
| **render.yaml** | Blueprint de Render: servicio web Docker con base PostgreSQL gestionada y health check en /health |

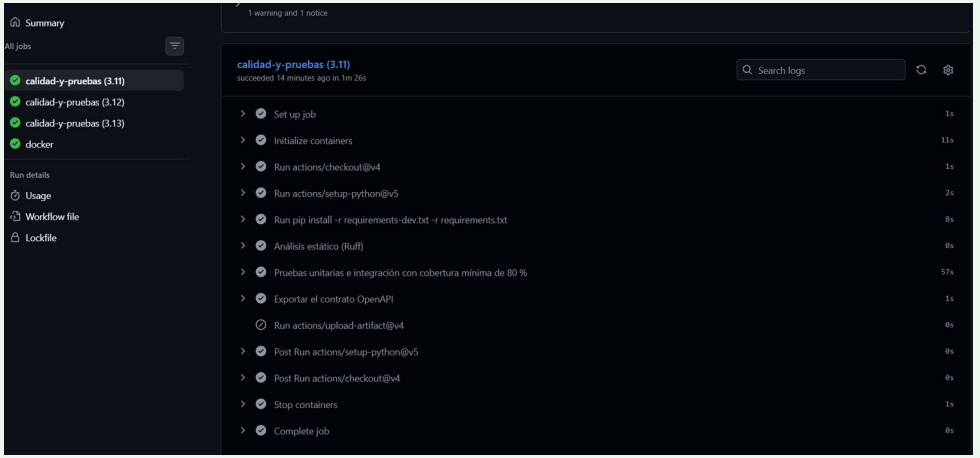
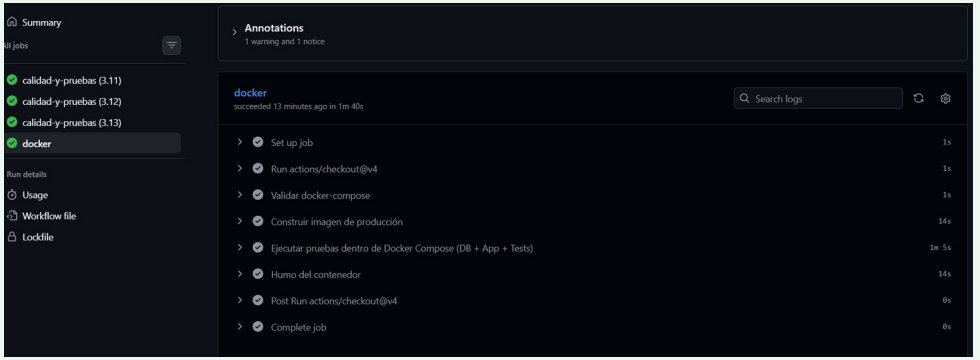
### Tabla 33. Comandos para la demostración

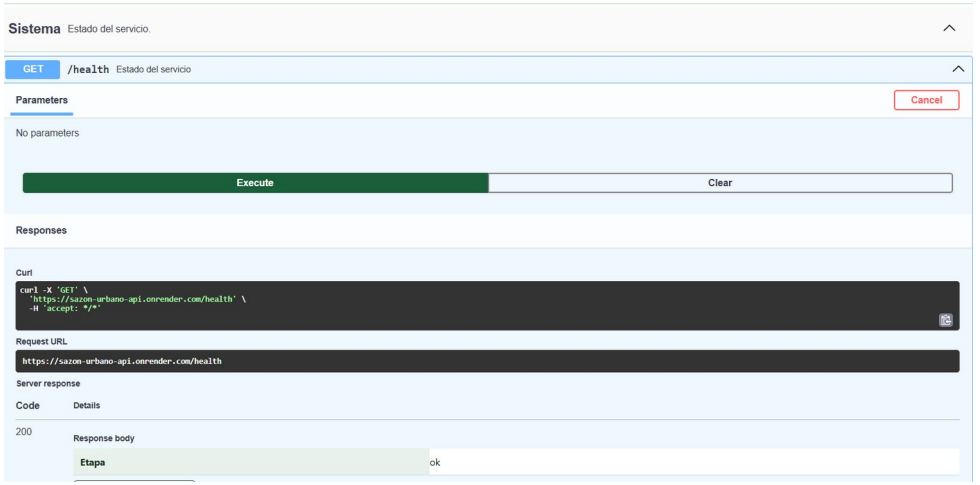
| Código |
|---|
| `docker compose up --build                    # PostgreSQL + API en http://localhost:8000/docs`<br>`docker compose --profile test run --rm tests   # Ruff + pytest con cobertura sobre PostgreSQL`<br>`docker compose down -v`<br><br>`pip install -r requirements-dev.txt`<br>`SEED_ON_START=true uvicorn src.api.principal:app --reload`<br>`pytest --cov=src --cov-branch --cov-fail-under=80` |

### Tabla 34. Despliegue en Render (pasos)

| Paso | Acción |
|---|---|
| **1** | Subir el repositorio a GitHub (git remote add origin …; git push -u origin main). |
| **2** | En render.com: New, Blueprint, elegir el repositorio. Render lee render.yaml y crea el servicio web y la base PostgreSQL. |
| **3** | Esperar el build. La variable DATABASE_URL se conecta sola y SEED_ON_START=true carga los datos del docente. |
| **4** | Abrir https://sazon-urbano-api.onrender.com/docs y comprobar /health. La URL está en el README y en este reporte. |
| **5** | Opcional: definir API_KEY para proteger la demostración y enviarla en X-API-Key. |

| Evidencia | Cómo obtenerla | Captura |
|---|---|---|
| **Pipeline de GitHub Actions en verde** | Pestaña Actions del repositorio, última ejecución con los dos trabajos en verde |  |

| Evidencia | Cómo obtenerla | Captura |
|---|---|---|
| **Docker Compose con DB, App y Tests** | Trabajo docker del pipeline en GitHub Actions: build de la imagen, pruebas con Compose y prueba de humo |  <br>  |

| Evidencia | Cómo obtenerla | Captura |
|---|---|---|
| **Despliegue público** | Navegador en la URL pública /docs |  |

## 12. Uso responsable de la IA y limitaciones

La IA ayudó a estructurar el código, redactar las pruebas y los documentos. Cada resultado se ejecutó y verificó: 269 pruebas, cobertura medida con pytest-cov y Ruff.

### Tabla 35. Registro del uso de IA

| Uso | Verificación | Decisión del equipo |
|---|---|---|
| **Generar el esqueleto por capas y el dominio** | Pruebas unitarias del dominio y ejecución de punta a punta | Se conservaron los nombres y reglas de las Unidades 2 y 3 |
| **Ampliar los criterios Gherkin originales con escenarios de error** | Cada historia se enlaza con una prueba real | Se conservaron los enunciados originales de la Unidad 2 |
| **Interpretar acciones del CSV de transiciones** | Se documentó la equivalencia en la sección 8 | Confirmar con el docente la definición de completar |
| **Escribir la configuración de Docker, CI y Render** | Validación de sintaxis YAML; Docker y Render no se pudieron ejecutar en el entorno de desarrollo | Ejecutados en GitHub Actions y Render; evidencia en la sección 11 |

### Tabla 36. Limitaciones declaradas

| N.º | Limitación |
|---|---|
| **1** | Los criterios Gherkin originales son uno por historia; los escenarios de error son una ampliación del equipo para poder probarlos. |
| **2** | La autenticación por encabezados no es criptográfica (ADR-09). |
| **3** | Docker Compose no se ejecutó en el equipo local (Docker no está instalado); se verificó en el trabajo docker de GitHub Actions. El pipeline y el despliegue en Render sí se ejecutaron. |

| N.º | Limitación |
|---|---|
| **4** | La prueba sobre PostgreSQL solo corre con TEST_DATABASE_URL, por lo que en la medición local aparece como omitida. |
| **5** | Seguimiento del repartidor fuera de alcance (SP-07). El suscriptor de ventas es síncrono en memoria; en producción convendría una cola. |

## 13. Conclusiones

**Tabla 37. Conclusiones del equipo**

| Conclusiones |
| --- |
| El proyecto convierte el BRD y las historias de Sazón Urbano en una API REST cuyas reglas viven en un dominio sin dependencias externas: un pedido no se confirma si el cliente está inactivo, no está autorizado, no tiene líneas o no hay existencia, y si algo falla el inventario queda intacto. La máquina de estados, las Specifications y los eventos hacen que las reglas sean explícitas y fáciles de probar; la Unidad de Trabajo mantiene la consistencia entre pedido e inventario; y la concurrencia optimista protege la edición simultánea. El paquete del docente sirvió como conjunto de pruebas independiente: 120 casos y 20 transiciones coinciden con un oráculo calculado solo desde los CSV, sin reemplazar los requerimientos propios. La cobertura de 98.5 %, el análisis estático y el CI hacen que un cambio que rompa una regla sea rechazado antes de llegar a producción. Aprendimos que un diseño por capas cuesta más al inicio, pero permitió probar todo el dominio (90 pruebas) sin base de datos ni HTTP, cambiar de SQLite a PostgreSQL sin tocar el dominio y trazar cada regla hasta su prueba. Como trabajo futuro quedan la autenticación real con JWT, el seguimiento del repartidor, una cola para actualizar las ventas y medir la frecuencia de conflictos de versión. |